Design layer

the whole layer, as one document

Contents

1 Saga 4

1.1 Foundation 4
1.2 System at a glance 4
1.3 Context boundaries 5
1.4 Verification gates 5
1.5 End-to-end walkthrough 6

2 Durability 9

2.1 Foundation 9
2.2 Pending updates 9
2.3 Durable execution model 9
2.4 Checkpoint and effect permission 12
2.5 Recovery rules 13
2.6 Storage and claim lifetime 14
2.7 Storage adapters and delivery 15
2.8 Durable time and capacity 17
2.9 Operating a durable execution 17
2.9.1 Admission and drive 17
2.9.2 Failure classes and caller action 18
2.9.3 Committed local database effect 19
2.9.4 Discovery and reading 19
2.10 Storage verification 19

3 Execution 22

3.1 Foundation 22
3.2 Pending updates 22
3.3 Workflow model 23
3.4 Typed authoring 24
3.4.1 Native transformations and diagnostics 25
3.4.2 Callback records, keys, and correlation 25
3.5 Coordinator internals 26
3.5.1 Value storage and soundness 26
3.5.2 Readiness and worker lifetime 27
3.6 Execution lifecycle 27
3.7 Step action lifecycle 28
3.8 Compensation and uncertainty 30
3.9 Outcome and settlement 32
3.10 Time and capacity 33
3.11 Local ownership and reporting 34
3.11.1 One invocation receiver 35
3.12 Observation and verification 35
3.12.1 Reactor oracle 36
3.12.2 Run-cost benchmark 37
3.12.3 Usage examples 37
3.13 Composition extension contracts 37
3.13.1 Value-dependent traversal and local continuation 37
3.13.2 Approvals and historical review 37
3.13.3 Independent children and compensation authority 38
3.14 Runtime-authored graphs 39
3.14.1 Parametric node model 39
3.14.2 Publication and hostile input 40

Glossary 41

Saga

1 Saga

1.1 Foundation

Saga owns typed dependency execution and the evidence needed to settle effects.

NO-GOAL

A job-delivery or agent-policy runtime

Delivery, queue capacity, agent policy, protocol transport, and application retry policy belong to their respective owners.

NO-GOAL

Serialize closures or expose arbitrary native lookup

Executable closures are reconstructed from deployed definitions. Native dependency values cannot be obtained through runtime result names or caller-visible type erasure.

PRINCIPLE

Separate evidence from authority

An observed result, cancellation request, notification, or telemetry event cannot establish authority it does not carry. Compensation authority and absence of external effects require explicit contracts.

PRINCIPLE

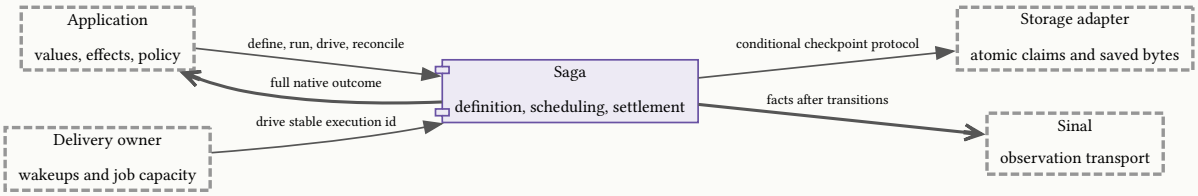
Keep package boundaries at distinct lifecycles

Saga owns effect progress and settlement. Storage owns persistent ownership, while applications compose delivery and agent lifetimes through small public ports.

1.2 System at a glance

One workflow model is interpreted by the local coordinator or by a checkpointed drive.

VIEWPOINT RUNTIME · ALTITUDE L1 · BOUNDARY Ownership around Saga



LEGEND



Storage implements a port; it does not choose workflow progress. Delivery acknowledgments and Sinal events never replace the saved outcome.

- The application supplies native input/output/error/undo-error records and callbacks. The `execution` context retains them through `Port` connections and `Outcome` reports.
- A `Local run` belongs to its starting process; owner exit requests cancellation. A `Durable execution` belongs to its saved record; driver exit stops driving without recording cancellation.
- The package requires Sinal and standard Gleam/OTP/time/JSON libraries. PostgreSQL, Blueprint, Grind, Fabric, Relay, and provider clients compose through optional adapters or application code.

Saga

responsibility

Execute typed dependencies and preserve effect evidence.

interface

Workflow authoring; local execution; optional durable capability; storage, outcome, reporting, observation, and testing ports.

interactions

Calls application effects and recovery callbacks; interprets storage conditional writes; emits observations through Sinal.

invariants

One native type relation per Port; bounded per-run admission; uncertainty and settlement survive projection boundaries.

failure

Separate definition/configuration rejection, workflow outcomes, durable operational failures, and lost report delivery.

1.3 Context boundaries

Execution defines what a workflow does; durability defines how its progress survives.

Context	Owns	Depends on
execution	Workflow definition, typed ports, scheduling, retry, compensation, undo, outcome, settlement, local ownership, telemetry, and the intended composition and runtime-graph extensions.	Application callbacks and Sinal.
durability	Saved execution identity, checkpoints, compatibility, recovery resolvers, claims, the storage port, reference adapters, and delivery obligations.	The Workflow, EffectKey, and Outcome values of execution.

- Durability is the customer of execution: it attaches codecs, versions, and resolvers to the same Workflow and never defines a second graph. The same Workflow is interpreted by the local coordinator or by a checkpointed drive.
- Execution owns every term about a step action and its settlement. Durability owns every term about saved state and its ownership. Neither context redefines the other's terms; each cites the owning slug.
- The PostgreSQL storage adapter is a nested project with its own layer under `integrations/saga_postgres`. It implements the durability storage port and owns SQL, schema, and database-clock leases.

1.4 Verification gates

One registry of profiles proves each obligation by the evidence that can establish it.

Evidence	Guarantee checked	Boundary
examples/order_consumer	Common and advanced native use, mapped errors, failures and undo, durable recovery, Blueprint bridge, and full reporting.	Separate package using public modules only; each documented usage topic is one compiled module that asserts what it shows (ADR-0014).
fixtures/negative with positive controls	Opaque Port, EffectKey, and Execution; wrong error, choice, and Continue types.	Actual compiler failures with expected diagnostics.
Scheduling, lifecycle, rollback, retry, unknown-effect, and reporting tests	Admission, timers, owner loss, reverse undo, evidence, and mailbox cleanup.	Public behavioral and process probes.
scripts/check_durable_restart.sh	Concurrent effect and compensation receipts, and stable-key reconciliation.	Actual VM kill and fresh definition in another VM.
Reactor oracle	Seven scoped scenarios; reverse undo and sibling settlement are deliberate differences.	Reactor 1.0.6; not full Reactor API parity. Pin, mapping and limits: oracle reference.
Benchmark harness	Build-once, store, and readiness cost, with correctness checks.	Measured observation; no latency guarantee. Method and results: benchmark reference.

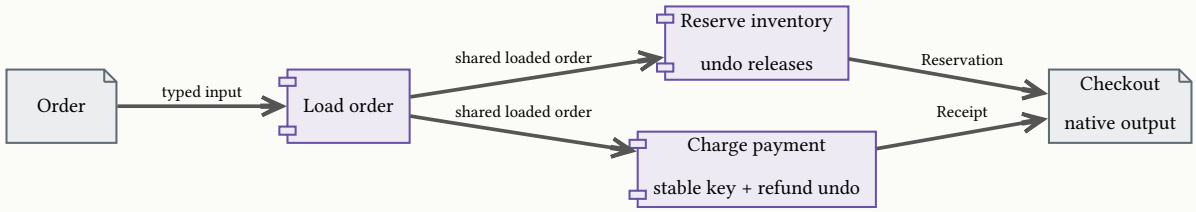
Profile	Obligations and evidence
fast	Formatting, Ruff correctness lint, ShellCheck, actionlint, gate regressions, and the root strict build and tests.
ci	All four packages; external order consumer tests and executable; strict benchmark build; disposable PostgreSQL 16; positive and negative compiler fixtures; actual VM restart probes.
design	Render freshness, vocabulary, links, and integrity of both native layers.
full	Every ci and design obligation.
oracle	Seven real Reactor 1.0.6 scenarios, captured upstream output, and normalized Saga comparison including the checked deliberate differences; full root tests.
benchmark	The chain, fan, and wide harness with correctness assertions and observational timings; no latency threshold.

- Every package build treats Gleam warnings as errors and compiles authored Erlang in `src` and `test` with `erlc -Werror` and dependency includes. Generated dependency code and retained oracle or fixture bytes are excluded from authored-script formatting and lint.
- Gate commands check the tree without repairing it. A profile stops at its first failure and retains per-check logs, nonempty result entries, dependency revisions, and environment metadata under `.artifacts/PROFILE`.
- Hosted verification checks out immutable sibling revisions from `sibling-revisions.txt` without persisting credentials. Fork pull requests run the same required checks.
- A push, pull request, or manual run requires the ci and design profiles. The oracle runs on relevant source changes, weekly, and manually; the benchmark runs weekly and manually.
- The oracle proves only its seven scoped scenarios, not general Reactor or Temporal parity. The database harness disables fsync and synchronous commit, so passing runner and VM recovery tests establish no database power-loss durability. The decision is ADR-0013.

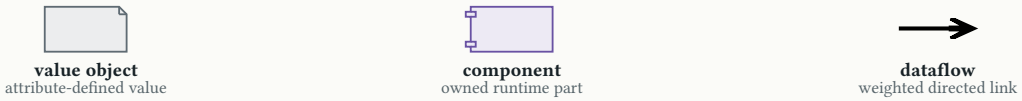
1.5 End-to-end walkthrough

An order reserves inventory and charges payment while preserving uncertainty and cancellation authority.

VIEWPOINT DATAFLOW · ALTITUDE L3 · COMPONENTS One typed order workflow



LEGEND



The external order consumer supplies native Order, Reservation, Receipt, PayError, and UndoError. Load runs once while independent branches run within the selected capacity.

- Define the graph once at application startup. Pure configuration sets concurrency and distinct attempt/deadline/settle/cleanup bounds before one local run starts.
- A clean charge and reservation produce native Checkout. A known payment decline fails with the typed business error; the inventory undo succeeds or contributes its located native UndoError.
- A MaybeCharged error marked by unknown_when can request a keyed retry. Even if retry succeeds, the earlier unknown attempt remains visible in CompletedWithUnknownEffects. Without a settling decision, default Hold retains inventory for reconciliation.
- For persistence, attach versioned root/step codecs and effect/undo/compensation resolvers to the same graph. start_or_reconnect stores one id/input before drive; a repeated compatible request reuses that execution.
- If the runner dies after the provider receipt but before checkpoint outcome, the next compatible drive invokes the resolver with the original key. Completed restores the receipt; NotSent proves replay permission; MaybeSent suspends.

- Explicit durable cancellation records intent, reconciles admitted work, and undoes only known completions. Driver/job death alone preserves resumable state. A survivor reads the saved full report or receives local owned-report settlement.
- The application retains the typed Outcome for audit/recovery and chooses outcome.classify for public behavior. Safe summary keeps settlement facts without exposing payloads; reporting operational error stays separate from obtained workflow failure.

CONTEXT

Durability

2 Durability

2.1 Foundation

Durability owns saved execution progress, claim-fenced storage, and recovery under explicit evidence.

GOAL

Recover saved execution without guessing effect absence

Optional persistence retains progress and stable identities across runner or VM loss. Recovery uses explicit application evidence before repeating admitted work.

INVARIANT

Durable effect permission follows a committed state

Admission and checked input are saved before a worker receives permission for the external action. A refused commit stops further dispatch.

mechanism

INVARIANT

A current claim guards every checkpoint write

The adapter checks claim identity, cancellation observation, and revision atomically. A released or replaced claim cannot write progress.

mechanism

PRINCIPLE

Resolve interrupted effects by evidence, never by replay

Only explicit application evidence permits repeating an admitted action whose result is unsaved. Losing a driver, a runner, or a delivery never establishes that an effect did not happen.

2.2 Pending updates

ruling **Checkpoint migration and delivery transaction** 2026-10-06

Explicit definition and checkpoint migration, and any optional Saga/Grind runner or outbox, await a decision. An application owns wakeups, transaction boundaries, and redelivery; no generic durable runtime is inferred from similar claims or durations.

ADR-0004

ruling **A total drive return deadline needs an owner decision** 2026-10-06

The drive execution budget is followed by synchronous runner drain and possible bounded claim release. A stronger total-return promise requires an explicit cleanup owner and takeover contract.

ADR-0009

verify **Durability timing margins and recovery rules are unchecked** 2026-10-17

The timing margins, the stable-identity rules and the committed-local-effect recovery rules came from the retired durability guide. The compiled durable-run examples exercise stable identity, reconnect, unfinished-run discovery and committed-local-effect recovery. The timing margins need a comparison with source.

2.3 Durable execution model

A saved execution outlives a drive; a fresh compatible definition supplies executable behavior.

Saved status pending suspended finished

Durable execution

aggregate · stateful · workflow · owned by Saga durable protocol

The storage-addressed aggregate that retains input, progress, compatibility, and outcome.

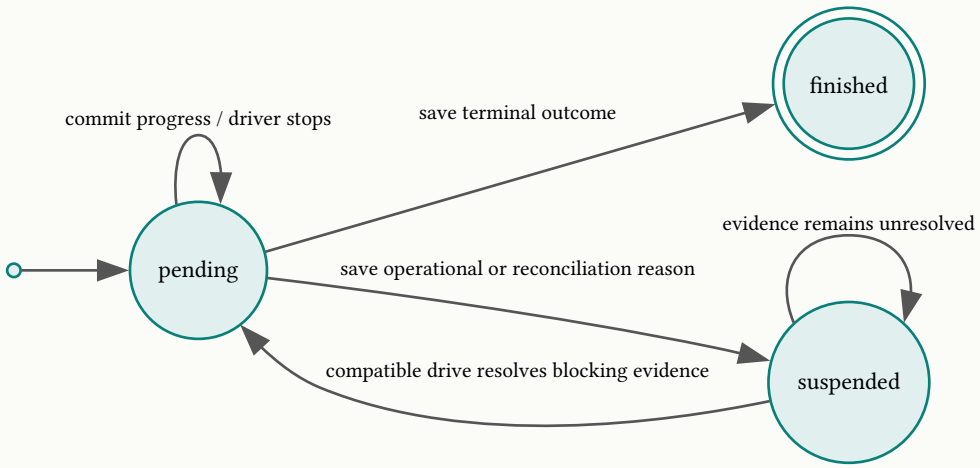
ATTRIBUTES

- authored **Execution identifier** · Caller-chosen id within Storage
  A stable application key normally includes workflow identity. It is independent from local runner and correlation identity.
- derived **Compatibility and input** · Computed stamp × checked encoded native input
  Reconnect requires compatible definition and exactly the same encoded input.
- derived **Status** · Saved status saved-status · saved-lifecycle
  One latest committed authority; a suspended reason retains typed failure details.
- authored **Cancellation intent** · Monotonic requested flag
  Independent from status and claim; a terminal commit must observe its current value.
- derived **Checkpoint revision** · Increasing Revision
  Changes only on accepted checkpoint commits, not cancellation alone.
- derived **Persisted correlation** · Absent before first drive, then one Sinal Correlation
  The first drive chooses and commits it; every later handle uses the saved value.

RELATIONSHIPS

- `1 : 0..1` Has one current Claim; lack of a runner does not make it cancelled.

STATE MACHINE The saved execution lifecycle



Cancellation is an independent intent axis. Driver timeout, death, or lost ownership leaves the latest saved state authoritative and does not introduce cancellation.

Persistence capability

responsibility

Check an existing Workflow's eligibility and restore-compatible identity.

interface

`durable.new` with labeled root codecs; with_version, with_config, with_max_checkpoint_bytes; step recoverable/restore_undo/ resolvers.

interactions

Uses step versions, graph wiring, and codec identities to compute the stamp; attaches no second graph.

invariants

Every persistent step is recoverable; every compensating step declares pure undo reconstruction including explicit NoUndo. Compatibility precedes application decoding.

failure

Missing/empty persistent capabilities are source defects and new panics with all problems. Changed stamps return IncompatibleDefinition before decoder callbacks.

Durable handle and driver

responsibility

Create/reconnect one saved execution and advance it under a current claim.

interface

`start_or_reconnect`, `reconnect`, `read`, `cancel`, `drive`, `unfinished`; growing Error with stable error_kind.

interactions

One Storage serves all ids; drive claims, restores, runs, saves outcome, stops helpers, and releases.

invariants

Create/reconnect runs no effects; same id/definition/encoded input is idempotent. Finished executions return saved Outcome without another effect.

failure

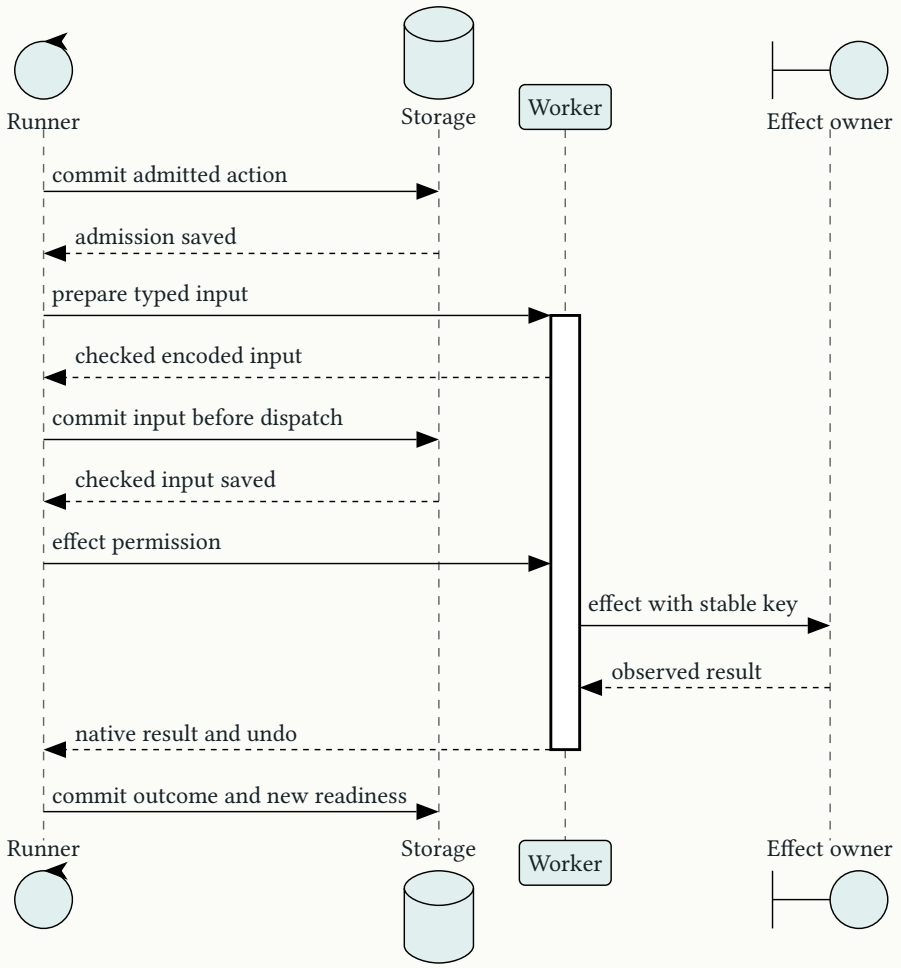
Busy, Transient, NeedsReconciliation, Incompatible, and Defect remain separate classifications; SuspensionNotSaved retains distinct original and recording failures.

- One optional persistence model and the delivery boundary are recorded in ADR-0004.

2.4 Checkpoint and effect permission

Admission, checked input, and effect outcome are separate durable boundaries.

SEQUENCE Commit before effect permission



Loss after permission but before the accepted outcome creates an interrupted admitted action. The stable key and resolver must establish whether its effect happened.

Checkpoint interpreter

responsibility

Save complete execution values and reconstruct compatible run state.

interface

Opaque storage bytes; versioned envelope, typed checkpoint problems, and root/step codec boundaries.

interactions

Serializes step progress, inputs/outputs, selected decisions, retries/deadline, failure/rollback intent, journal order, settlement, and uncertainty.

invariants

Never saves executable closures, PIDs, monitors, sockets, timers, worker leases, or credentials. Every commit rewrites a complete bounded snapshot.

failure

Malformed/foreign/graph-mismatched state, insufficient restored concurrency, missing compensation input, or nonrestorable undo fails closed.

- The compatibility stamp includes workflow name/version, ordered occurrences, addresses, dependencies/choice wiring, step versions, codec versions, attempt/time budgets, and declared recovery capabilities. Runtime node ids do not enter it.

- Caller changes to callback meaning require version changes even with unchanged native types. A stamp checks declared identity, not semantic equivalence of arbitrary closures.
- Codec encoding checks that produced text decodes; it does not prove semantic equality or determinism. The codec author owns those laws and any external resources captured by callbacks.
- Undo reconstruction must be pure and may run during validation and restoration. A Continue that carries undo cannot commit if declared reconstruction returns NoUndo.
- Codec attachment, undo/resolver configuration, and error mapping preserve recovery capabilities across modifier order. A Failed resolver added after error mapping needs a compensation decider in the mapped vocabulary or fails DeciderMissingAfterMapping.
- Checkpoint format 2 retains correlation. Format 1 opens with correlation.from_key(id); the next commit uses format 2. Earlier binaries cannot read the newer envelope; no automatic arbitrary-definition migration is implied.

2.5 Recovery rules

Only explicit evidence authorizes replay of an interrupted admitted action.

Recovery resolvers

responsibility

Establish results or proven absence after interruption without repeating uncertain callbacks.

interface

Attempt and undo Evidence: Completed, Failed, NotSent, MaybeSent; compensation resolver: Some(Recovery) or None.

interactions

Receive retained native input/output and the original EffectKey; applications query downstream receipts or enforce idempotency.

invariants

Callbacks must be repeat-safe. Known saved success is reused; NotSent alone authorizes replay; MaybeSent suspends.

failure

Missing or unresolved reconciliation saves RecoveryRequired with address, action, and key. Original compensation callback is never blindly replayed.

Saved boundary	Recovery action	Constraint
Success committed	Decode and reuse	No invocation of the completed step.
Admitted before checked input	Prepare again	No effect permission was issued.
Checked input admitted without result	Attempt resolver	Completed/Failed restore result; NotSent permits original-key replay; MaybeSent suspends.
Undo admitted without result	Undo resolver	Missing/MaybeSent suspends; NotSent permits saved undo; native undo failures retained.
Compensation admitted without decision	Compensation resolver	Some applies original-budget decision; None suspends; never repeats original decider.
Marked uncertain returned error	Retain unknown evidence and saved decision	Uncommitted attempt uses effect resolver; interrupted decision uses compensation resolver.
Cancellation already recorded	Reconcile admitted actions before rollback	NotSent does not start a new effect during cancellation.

BEHAVIOR

A saved success is reused after runner loss

Given A step's successful output was committed before the runner stopped.

When A compatible drive resumes the execution.

Then The saved output is reused and the successful step does not execute again.

level: boundary · area: Recovery

BEHAVIOR

Unknown admitted work cannot silently replay

Given An admitted action has no saved result.

When A drive resumes it.

Then Explicit resolver evidence determines its result or permits replay by proven absence.

Then Unresolved evidence suspends without repeating the external action.

level: boundary · area: Recovery

- Restored max_concurrency must accommodate saved in-flight attempts and compensation decisions. A smaller setting returns ConcurrencyBelowInFlight rather than pretending those admitted actions vanished.
- A storage/codec/size/refusal failure stops workers and dispatch. If suspension recording also fails distinctly, both causes survive; otherwise the last accepted checkpoint remains authority.
- Cancellation becomes visible at the next checkpoint and may therefore include a current action's timeout. A cancellation flag changed before admission or terminal commit wins the conditional-write race.

2.6 Storage and claim lifetime

The adapter supplies atomic revocable ownership; Saga supplies workflow transitions.

Claim status unowned · owned

Execution ownership

entity · stateful · workflow · owned by Storage adapter

The revocable ownership state for one saved execution.

ATTRIBUTES

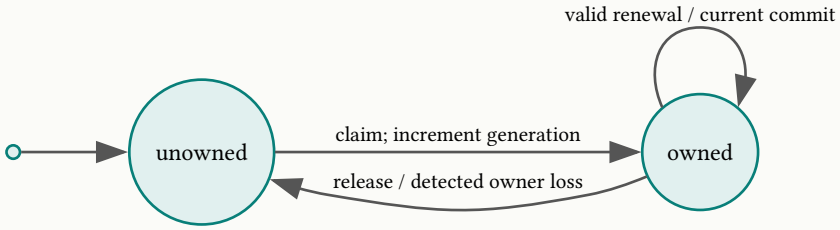
derived **Status** · Claim status claim-status · claim-lifecycle
Vacant ownership permits a new claim; adapter-supported owner-loss detection can revoke a current claim.

derived **Claim identity** · Execution id × generation × token
New claims increase generation and obtain a distinct adapter proof. A token alone does not authenticate an external business caller.

RELATIONSHIPS

1 : 1 Guards checkpoint changes of one saved execution.

STATE MACHINE Current ownership can be revoked



Lease expiry makes takeover possible under an adapter's policy. It cannot retract an external call already sent by an earlier runner; fencing guards saved writes.

Storage port

responsibility

Persist atomic execution bytes, claims, cancellation intent, and discovery.

interface

Storage.new supplies create/load/claim/commit/release/cancel/unfinished; optional renewal and operation Duration.

interactions

Returns opaque Stored and Claim values; Saga commits expected revision and observed cancellation with phase/data.

invariants

Ownership is the current claim value, not the committing process. Each refused operation leaves checkpoint bytes/revision unchanged.

failure

StaleOwner precedes CancellationChanged, which precedes Conflict; Busy excludes a competing live claim; unavailable, corrupt, timed-out, and missing records stay distinct.

Operation	Accepted effect	Refusal or idempotence
create	Revision/generation zero; uncancelled Pending bytes	AlreadyExists; never overwrites another execution.
load	Read latest bytes/revision/generation/ cancellation	NotFound or Corrupt distinct from availability.
claim	Advance generation; return Claim and Stored	Busy while live claim excludes competition.
commit	Current claim + seen cancellation + revision → new bytes/phase and revision+1	Ordered StaleOwner → CancellationChanged → Conflict.
release	Remove current ownership	StaleOwner for another claim.
cancel	Set cancellation flag without changing bytes or revision	Idempotent; durable cancel is a no-op for saved Finished.
unfinished	Up to limit unowned Pending/Suspended ids	Finished excluded; oldest first where adapter can tell.

- A heartbeat linked to the durable runner renews lease-based storage independently of checkpoint traffic. Retry backoff or a long attempt cannot depend on commit-only renewal.
- StaleOwner on renewal stops the runner. Unavailable/TimedOut renewal retries at the next interval; all later writes remain fenced if the lease expires meanwhile. A wrapper rebuilding Storage must preserve renewal.
- Drive uses a watchdog for synchronous runner storage operations; other durable operations use bounded helper calls. Timeout stops waiting and kills the helper/runner but does not itself prove whether a remote store applied an operation.
- Storage claim constructors expose adapter proofs but cannot establish validity alone. The adapter enforces current generation/token and its declared deployment scope at every mutation.
- The protocol decision is ADR-0005. Adapter-specific persistence and distribution guarantees remain separate from the core contract.

2.7 Storage adapters and delivery

Reference adapters differ in failure scope; delivery remains an application lifetime.

Memory storage

Keep durable records within one supervised or linked VM actor.

responsibility

Serialize the Storage protocol in one actor and monitor claiming process loss.

interface

start, supervised(name), named(name), stop, and storage(handle).

interactions

One actor serves all ids; named handles locate a restarted actor.

invariants

Current claim may be used by any process; claiming-process monitor controls owner loss. Restart discards all saved executions.

failure

Five-second operation wait returns TimedOut; stopped/missing actor returns Unavailable. Runner loss can be survived; VM/store loss cannot.

Reference file storage

Recover checkpoint bytes in a fresh VM with one directory owner.

responsibility

Apply atomic per-execution mutations using canonical-path locking and synced temporary-file replacement.

interface

Pure file.open(canonical_absolute_directory) returns Storage; caller creates and owns directory.

interactions

Erlang FFI reads/writes records; a claim records this VM's claiming process. Earlier-VM claims are unowned at recovery.

invariants

One VM at a time uses a canonical directory; lock wait is bounded. Atomic replacement preserves complete snapshots.

failure

Malformed/read/write/lock failures remain typed. Concurrent VMs, path aliases, and power-loss directory durability are outside its guarantee.

PostgreSQL persistence adapter

responsibility

Implement Storage across VMs with database-backed fenced claims.

interface

The separate saga_postgres package's public configuration/storage/migration port; caller-owned pool.

interactions

Saga owns heartbeat/drive; the adapter owns database clock, lease, schema, transactions, and generation/token checks.

invariants

No second pool is implicitly allocated by Saga; core Saga gains no database dependency.

failure

Lease loss permits recovery under the adapter's contract; external-effect uncertainty still belongs to Saga/application reconciliation.

- Detailed PostgreSQL design belongs to `integrations/saga_postgres/docs/design/design.typ`. The core owns only the persistence port and its lifetime obligations.

Delivery integration

responsibility

Wake stable executions and map delivery interruption into explicit Saga operations.

interface

Application-owned stable id, reconnect/read/drive/cancel, unfinished scanning, and error_kind mapping.

interactions

A Grind job supplies capacity and wakeup; Saga's saved outcome supplies workflow completion authority. Fabric supplies parent policy and continuation.

invariants

Driver stop is resumable interruption; cancel is recorded intent. No automatic full-local-workflow retry follows redelivery.

failure

Busy/Transient can be retried deliberately; NeedsReconciliation needs operator/application evidence; Incompatible/Defect require correction. Atomic job admission needs an application transaction/outbox.

- The checkout application selects Grind cancellation while driving Saga and explicitly calls durable.cancel when cancellation is intended. Stopping drive alone must leave the execution recoverable.
- Delivery deadlines reserve time for Saga stop, drain, release, settlement, and any persistence owner-loss window. Retrying a completed failed workflow is an explicit application policy.
- An application using a shared pool starts migrations and stops infrastructure under its own lifecycle. Store reuse does not merge job, agent, and workflow claim authority.

2.8 Durable time and capacity

Each durable bound limits one operation; none promises a total return time.

Bound	Default	Ownership and limit
Storage operation	5 seconds, then TimedOut	`with_call_timeout`; minimum one millisecond; a slow call suspends and stops the runner.
Memory adapter call	5 seconds	Not configurable.
File adapter mutation lock	5 seconds, then Busy	Not configurable.
Checkpoint bytes	16 MiB (16 777 216 bytes)	Whole saved snapshot; `with_max_checkpoint_bytes` changes it; an oversize snapshot returns CheckpointTooLarge.
Drive execution	Required caller Duration	Positive; after expiry, runner drain and claim release precede return.
Drive caller exit	Runner stops	The checkpoint is kept and no cancellation is recorded.
PostgreSQL claim lease	30 seconds, renewed every 10 seconds	Owned by the adapter's own layer.
Conformance owner-loss window	Declared by the adapter	Passed to the Conformance suite as the owner-loss allowance.

- A drive starts its execution-budget clock after spawning and monitoring the runner. Its drain receives wait up to five seconds each and reset after a claim or result message; the finite protocol has at most one claim and one result. An abnormal exit with a known claim then performs one separately bounded release call.
- A drive may therefore return after its requested Duration. Drain scheduling and release timing are margin obligations when an application nests a drive under a job worker deadline. The decision is ADR-0009 a stronger total deadline is a pending ruling.
- The storage call timeout, lease, renewal interval, effect timeout, and delivery retry policy must be compatible. A lost VM can leave Busy until the adapter detects owner loss, so a retry window must outlast the adapter's owner-loss policy.

2.9 Operating a durable execution

The application owns identity, wakeups, and evidence; Saga owns the saved protocol.

2.9.1 Admission and drive

- The application chooses one stable execution id before job admission or external delivery. Reuse of an id requires the same compatible definition and the same encoded input; another input returns InputMismatch.
- `durable.new` checks that every step is recoverable and panics naming each offending step and codec, because that is a source defect. The workflow version defaults to "1"; `with_version` changes it whenever the workflow's behavior changes, so a saved execution of the old behavior returns IncompatibleDefinition.
- A drive claims the execution, restores it, runs, saves the outcome, stops its helpers, and releases the claim. Timeout, caller exit, and runner loss each stop the runner: in-flight attempts are killed, the claim is released at once, and the next compatible drive resumes from the last checkpoint by asking each interrupted attempt's resolver. Only the loss of the runner's VM leaves the claim to the adapter's expiry. Only `durable.cancel` cancels.
- Correlation is set before the first drive, which saves it. The caller inspects the full typed Outcome, because a returned report can describe failure, cancellation, or unresolved effects.

BEHAVIOR

A repeated admission reuses the saved execution

Given An execution is saved under an id.

When The same id and the same encoded input are admitted again.

Then The saved execution is attached and no effect runs.

Then A different input for that id is refused as an input mismatch.

level: boundary area: Admission

BEHAVIOR

Drive interruption preserves resumable state

Given A drive stops because its budget expires, its caller exits, or its runner is lost.

When A compatible drive later resumes the execution.

Then The execution continues from its last checkpoint and no cancellation is recorded.

level: boundary area: Admission

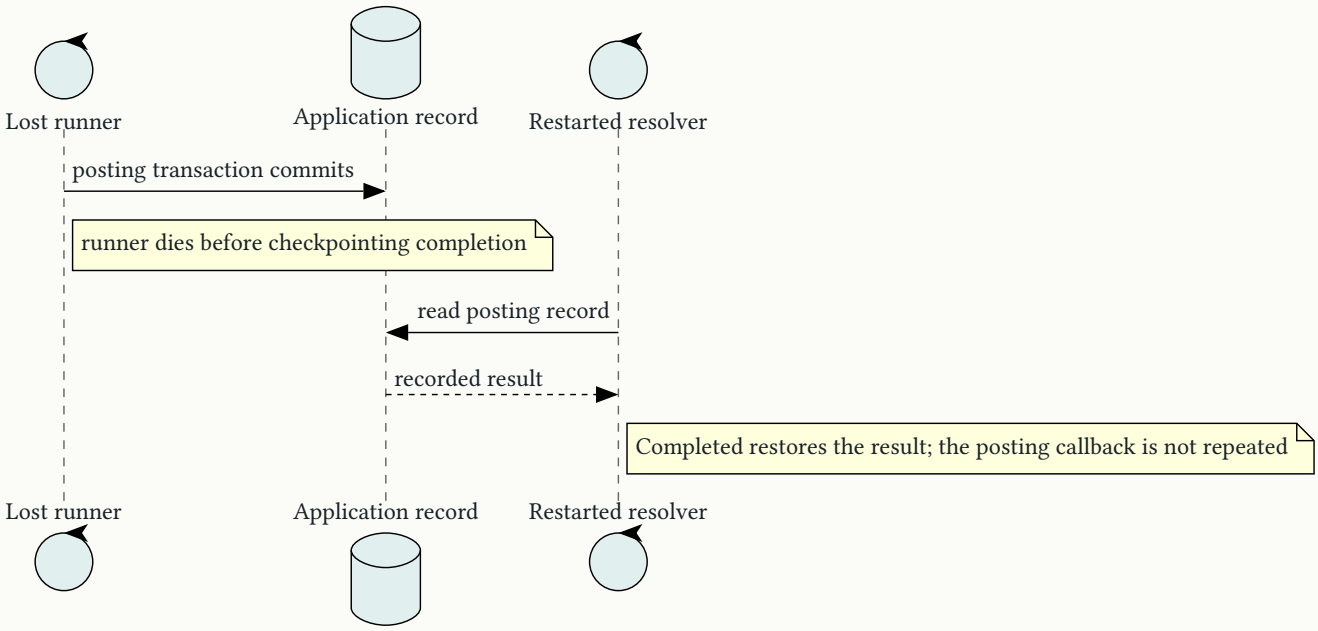
2.9.2 Failure classes and caller action

Error kind	Application action
Busy	Retry no sooner than the adapter's owner-loss window, under the application's delivery policy.
Transient	Retain the stable id and retry deliberately; inspect the saved state before assuming work absent.
NeedsReconciliation	Establish downstream effect evidence or request operator attention.
Incompatible	Supply the matching definition and input, or use an explicitly approved migration.
Defect	Correct configuration, codec, definition, checkpoint, or adapter behavior.

- Typed StorageFailure, CodecFailure, InvalidCheckpoint, CheckpointTooLarge, and RecoveryRequired detail stays available. SuspensionNotSaved retains both causes when recording the suspension fails differently from the original cause.
- NotSent requires proof of absence and MaybeSent preserves suspension. A remote effect is never replayed because an old driver disappeared or a delivery system retried.

2.9.3 Committed local database effect

SEQUENCE A committed posting survives runner loss



The application's posting record, not a missing checkpoint, establishes the result.

- Sharing a database or a pool does not make an application's SQL transaction atomic with Saga's later checkpoint. The adapter's commit boundaries keep those writes separate.
- The application keeps a stable business operation id and an authoritative posting record inside its own transaction, makes the effect idempotent, and makes it discoverable under that identity. A missing checkpoint is never evidence that SQL rolled back.
- The attempt resolver returns Completed only when the application record establishes the result. A failed lookup or inconclusive absence stays MaybeSent and authorizes no further effect.
- A delivery system that redelivers the coordinator resumes Saga recovery; it never authorizes resubmitting a payment. The application owns financial evidence, reservation, and posting independently of both libraries. The decision is ADR-0012.

2.9.4 Discovery and reading

- `durable.unfinished(storage, limit)` lists candidates without reserving them, and reconnect plus drive can still meet Busy. Saga supplies listing and reconnect, never a sweeper or delivery scheduler.
- `durable.read(run)` returns Pending, Suspended with its reason, or Finished with the saved report, without a live runner. The saved Outcome is the completion authority; notifications and telemetry are observations.

2.10 Storage verification

Protocol conformance and deployment durability need different evidence.

Storage conformance

responsibility

Expose extension checks for any storage adapter without private imports.

interface

`storage.conformance.run` accepts a fresh fixture per scenario, cleanup, an operation timeout, and the adapter's declared owner-loss allowance.

interactions

Consumer tests supply public storage and resolver fakes; fresh-VM probes exercise file recovery.

invariants

The suite checks atomic create, claims as values, live and lost ownership, stale writes, the cancellation race, listing, release, and killed-runner recovery.

failure

A located suite failure names the violated contract. A one-VM pass proves neither distributed fencing nor media durability, so adapters need separate deployment tests.

- The restart probe kills a VM during concurrent effect and compensation work, then recovers from fresh definitions and saved receipts in another VM. It stays in the verification set because no in-process test can establish that recovery.
- Evidence across both contexts appears in the root verification gates.

CONTEXT

Execution

3 Execution

3.1 Foundation

Execution owns typed dependency scheduling, effect evidence, and settlement.

GOAL

Execute native typed workflows with complete evidence

One definition supports shared dependencies, concurrency, retry, compensation, and undo while retaining caller-owned values and failure types.

INVARIANT

A native dependency preserves its producer type

An opaque Port binds one produced value type to every consumer. Only construction of that Port can establish its node identity and typed read capability.

mechanism

INVARIANT

An uncertain action remains recorded after recovery

A later retry, replacement, abort, or hold does not erase an earlier unknown effect. Completion with such evidence has a distinct outcome.

mechanism

INVARIANT

Only known completions enter the undo journal

An interrupted attempt is never reported as undone. Successful undo entries run in reverse actual completion order and retain every failure.

mechanism

PRINCIPLE

Preserve local capability without persistence ceremony

Native local execution requires no codecs or storage. Persistence attaches to the same definition through the durability context and checks its additional obligations.

3.2 Pending updates

build **Value-dependent composition and bounded traversal** 2026-10-06

Typed value-dependent composition and homogeneous traversal need ordering, graph-size, cancellation, and nested-capacity contracts. Arbitrary local typed fragments stay available; their exact admission and durable eligibility need a ruling before an API is accepted.

ADR-0008

ruling **Approval and historical fork authority** 2026-10-06

Durable approval waits, authenticated revision-checked deduplicated signals, and historical review forks need two owner decisions. One decision is how a changed answer treats prior effects; the other is which continuation owner suspends a Fabric agent.

ADR-0008

build **Independent children and completed-run compensation** 2026-10-06

Typed admission, attachment, observation, command reconciliation, journal retention, and exclusive compensation authority are designed and unbuilt. Scripted laboratory records establish the intended distinctions but not atomic admission, authorization, races, or recovery.

ADR-0008

build **Runtime graph publication and execution** 2026-10-06

Finite-contract registered graphs, parametric node instantiation, exact connection compatibility, publication checks, and execution are designed and unbuilt. Generic node families, native adapters, LLM nodes, visualization, and persistence remain required within this surface.

ADR-0008

build **Extended graph validation and introspection** 2026-10-06

Declared associative multiple-writer reducers, separate diagnostic labels and metadata, and graph visualization export are unbuilt. Pure authoring keeps working without OTP startup; static construction already excludes cycles and missing producers.

ADR-0008

ruling **Shared capacity and explicit lifecycle extensions** 2026-10-06

Shared or partitioned budgets across runs and children, coordinator supervision, explicit in-memory halt/resume, and undo retry policy await a decision. Concurrency is per run and each undo is attempted once.

ADR-0008

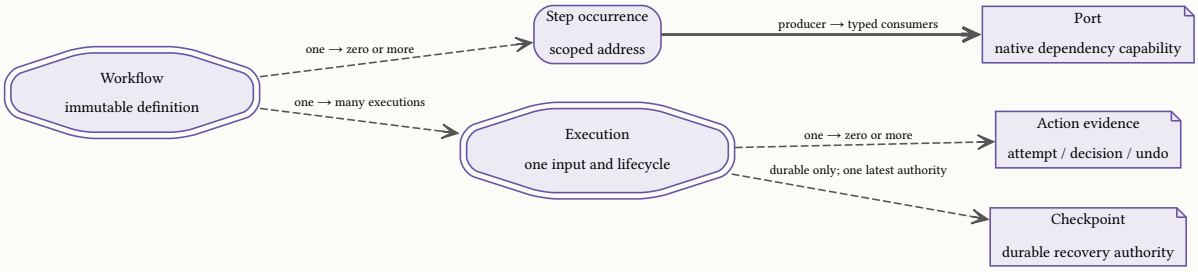
verify **Defaults and rules folded from retired guides are unchecked** 2026-10-17

The defaults and setters tables, the callback and correlation rules, and the unknown-effect semantics came from retired usage guides. The compiled usage examples exercise the observable topics. Only the attempt timeout default has been compared with source. Each other numeric default and bound needs a source check.

3.3 Workflow model

Definitions, execution identity, and observed action results are separate planes.

VIEWPOINT DOMAIN-MODEL · ALTITUDE L2 · CONTEXTS The model and its cardinalities



LEGEND



A step definition can be used in multiple occurrences. Runtime node numbers support scheduling but do not become stable persistence identities.

Workflow

aggregate · immutable · workflow · owned by Saga authoring

The validated reusable native dependency definition.

ATTRIBUTES

authored **Native contract** · Input × output × business-error × undo-error
Caller types remain generic throughout construction and execution.

authored **Definition name** · Diagnostic workflow name
Names the definition; it does not identify one execution.

derived **Step occurrences and dependencies** · Validated acyclic typed dependency structure
Derived once when the builder returns its output Port.

RELATIONSHIPS

1 : 0..n Defines step occurrences and supports separate executions.

Step and Port

value-object immutable workflow owned by Saga authoring

The operation contract and its construction-owned native dependency capability.

ATTRIBUTES

authored **Step policy** · Attempt × recovery × undo × timing × persistence capability
A Step can be configured without running its callback. Persistence is optional.

derived **Port membership** · Construction identity and typed fetch
A Port can only be read at the type paired with its producer. Foreign membership is rejected.

RELATIONSHIPS

1 : 0..n One produced Port may feed several consumers, sharing one scheduled producer.

Effect key

value-object immutable workflow owned by Saga execution

The opaque context of one attempt, compensation decision, or undo.

ATTRIBUTES

derived **Idempotency identity** · Execution × step address × action family
Stable across attempts of the same step and across durable restart. An undo has its own identity.

derived **Attempt identity** · Action identity × positive attempt number
Distinguishes attempt evidence; its text is opaque and must never be parsed.

derived **Correlation** · Sinal Correlation
Chosen at local start or saved at the first durable drive.

Value or sum	Meaning	Refinement
StepAddress	Scope, name, occurrence	Occurrence is positive; scope records embed/ choice structure.
AttemptFailure	Returned(error), Crashed(crash), TimedOut	Closed; preserves native error independently from effect status.
Undo	NoUndo or application action	NoUndo is absence, never a successful no-op.
Timeout	After(Duration) or Infinity	Positive millisecond resolution where required; explicit Infinity lifts the bound.
Recovery	Retry, RetryAfter, Continue, Abort, AbortAfterCleanupFailure, Hold	Closed; replacement includes its own undo capability.
Outcome	Completed, CompletedWithUnknownEffects, Failed, Cancelled, Unresolved	Evidence-bearing alternatives have different payloads.

3.4 Typed authoring

Construction establishes dependencies once; execution reuses the validated graph.

Definition builder	Typed composition

Bind typed dependencies and validate every constructed step.

responsibility

Evaluate the builder once and collect every constructed occurrence.

interface

`define` panics for source defects; `try_define` returns all DefinitionErrors; `name` and `describe` expose read-only metadata.

interactions

Mints construction membership and Ports; collects reachable steps from the output; checks unused constructed steps.

invariants

Empty names, invalid budgets/timeouts, foreign Ports, and orphan steps are rejected together. Construction invokes no step effects.

failure

Builder raises become a located definition failure; `define` reports the workflow and offending steps. Runtime-derived settings use `try_define`.

Compose values while retaining one run and undo journal.

responsibility

Connect `perform`, `both`, nonempty `all`, `map`, `embed`, and closed `choose`.

interface

Opaque Port and Workflow values; caller-owned error adaptation through `map_step_errors` and `map_errors`.

interactions

Each embed introduces a distinct scope; each choose builds both branches and schedules a decision. Shared dependencies remain shared.

invariants

Closed branches have the same output/error types; only the selected branch executes. `map_errors` reuses the graph and never reevaluates the builder.

failure

Compiler rejects mismatched outputs/errors; runtime definition validation rejects foreign/orphan construction. Output transformation crashes become OutputCrashed.

3.4.1 Native transformations and diagnostics

- `map` transforms a Port's fetched value; it is not a scheduled memoized step. It may execute in each consumer task; expensive or effectful work uses a Step.
- `all` requires a first Port and accepts further Ports of the same native type. The caller handles an empty collection explicitly.
- StepDescriptor contains address, dependency addresses, undo capability, compensation capability, attempt limit, and optional step timeout. It exposes neither callbacks nor executable control over child internals.
- Step names are diagnostic and structural identity, not native value lookup. Reassociation or different scopes can change durable identity even when the final business value remains equivalent.

3.4.2 Callback records, keys, and correlation

- A step callback receives one record. `step` receives the input only; `effect` receives the input and an opaque **Effect key** `undo` receives UndoRequest(input, output, key); `compensate` receives FailedAttempt(input, failure, attempt, attempts_left, key). Callers read records by label and the key through its accessors (idempotency_key, attempt_number, attempt_key, correlation_of).
- The **Correlation** of a run is mandatory and reaches every callback through its key. A local run takes a fresh unique correlation unless configured; a durable execution takes the saved one, or the key-derived default. A step's own clients attach it to join the run's observations.
- A local key belongs to one run: its **Idempotency key** derives from the run id, which each local run creates anew. A durable execution derives its keys from its execution id, so they survive restart.
- `define` reports every definition defect together and panics naming the workflow and each offending step. `try_define` returns every DefinitionError for definitions built from runtime data, and `describe_definition_error` renders one. `map_errors` and `map_step_errors` adapt error vocabulary; no saga-owned error type is ever required.

BEHAVIOR

A shared producer runs once

Given Several consumers depend on the same Port in one execution.

When The workflow executes.

Then The producer succeeds at most once under its retry policy, and every consumer reads that committed output.

level: boundary area: Authoring

BEHAVIOR

Unselected branch effects do not execute

Given The workflow contains a closed choice.

When The choice decision selects one branch.

Then Only that branch's effects execute; shared prerequisites remain shared.

Then Durable recovery preserves the selection before further branch effects.

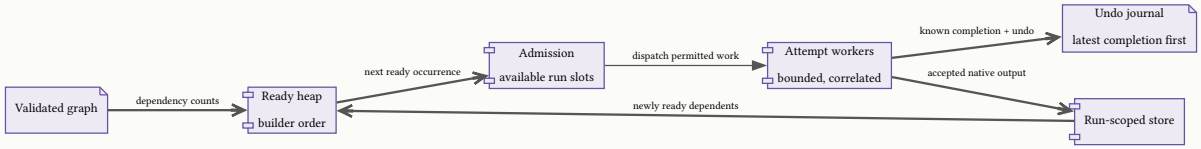
level: boundary area: Closed choice

- Authoring and scope semantics are owned here; the representation decision is ADR-0001. The build-once storage trade is ADR-0002.

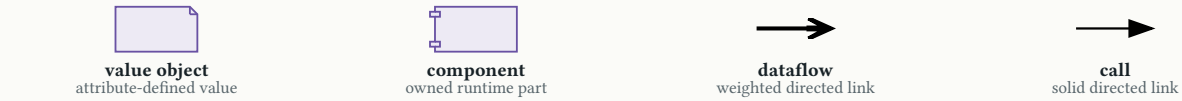
3.5 Coordinator internals

Run state owns values, readiness, workers, and the completion journal together.

VIEWPOINT DATAFLOW · ALTITUDE L4 · INTERNALS A coordinator advances one run



LEGEND



A durable commit gates dispatch across these transitions. The coordinator receives uniform action events while typed node closures own native values.

Coordinator

responsibility

Serialize transitions of one run and enforce readiness, capacity, timers, settlement, and rollback.

interface

Uniform completion/control messages; read-only progress; one terminal delivery function.

interactions

Interprets immutable node closures with a fresh store, monitors owner/workers, and optionally commits checkpoint values.

invariants

Only ready work enters a slot; attempts and compensation decisions share max_concurrency; retries reenter admission instead of bypassing capacity.

failure

A terminal trigger stops admission and initiates settlement. Durable storage/refusal or driver loss stops dispatch under the last checkpoint authority.

3.5.1 Value storage and soundness

Run-scoped store

responsibility

Retain completed native outputs and typed action records for one run.

interface

Internal opaque Store; node-bound put/fetch operations.

interactions

The Port's typed fetch and producer are created together; the coordinator threads a fresh store through run state.

invariants

No other run shares the store. Only the producer-bound native type reads a node's value; type erasure is confined to this module.

failure

Reading a value before dependency commit is an internal defect and panics. Opaque Port construction and compiler-negative fixtures exclude foreign typed reads.

- The implementation uses an opaque native carrier and an identity cast, never caller-visible Dynamic. Readiness and one binding of node identity to native type make that cast sound; ADR-0002 records the choice.

3.5.2 Readiness and worker lifetime

Readiness index

responsibility

Admit ready nodes without rescanning the whole graph.

interface

Dependency counts, dependents, ordered ready min-heap, active count, completed count.

interactions

A committed success lowers dependent counts; retry readiness returns to the heap.

invariants

No dependent fetch precedes committed output. Builder order breaks readiness ties, while actual completion order defines undo order.

failure

Waiting nodes become skipped after admission closes; stale timer/worker messages cannot advance another action sequence.

Action workers

responsibility

Isolate attempt, compensation, and undo execution from coordinator transitions.

interface

Correlated action sequence, result, monitor, timeout, and durable permission channel.

interactions

Attempts prepare typed input; persistence checks that input before effect permission; workers report returned result or typed crash evidence.

invariants

A stale completion cannot settle a newer action. A worker killed before permission cannot start its effect; a killed admitted action remains uncertain.

failure

Raises and exits become Crash; time limits kill workers and retain unknown-effect evidence. Detached durable runners stop their workers without inventing cancellation.

3.6 Execution lifecycle

Admission, sibling settlement, and reverse undo are distinct phases.

Execution phase running Admits ready work. settling Waits for active siblings without admitting new work. rolling-back Runs known undo entries sequentially. finished Retains one terminal outcome; coordinator exits.

Execution

aggregate · stateful · workflow · owned by Saga coordinator

One input interpretation of a Workflow and its effect evidence.

ATTRIBUTES

derived **Run identity** · Local fresh run id or durable execution id
Local id is fresh per start; the durable id is caller-chosen within one store.

derived **Phase** · Execution phase execution-phase · execution-lifecycle
Derived from accepted transitions; public progress exposes the three active phases only.

derived **Primary trigger** · Failure, cancellation, or held evidence
The original stopping cause remains separate from subsequently observed sibling and cleanup failures.

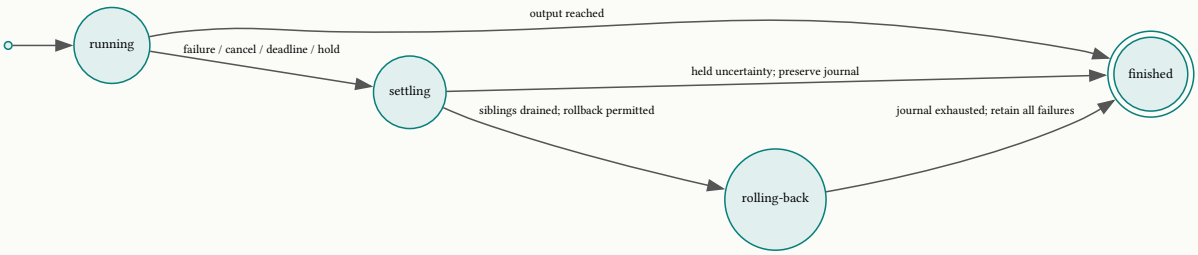
observed **Action evidence** · Known results, journal, and unknown actions
Accepted callback results record observations; recovery decisions supply explicit transition authority.

RELATIONSHIPS

1 : 1 Interprets one Workflow definition.

1 : 0..n Owns attempts, compensation decisions, and journal entries.

STATE MACHINE The execution phases



Cancellation after settlement has begun is idempotent and does not replace the stopping trigger. A completed last step can still be undone when cancellation wins before terminal completion.

From	Event and guard	Next state	Evidence or refusal
Running	Known output reached before a stopping trigger	Finished	Completed, or CompletedWithUnknownEffects.
Running	Failure, explicit cancel, local owner exit, deadline, or Hold	Settling	Skip waiting steps; preserve primary trigger.
Settling	Active work returns before settle bound	Settling	Accept known completions and retain later failures.
Settling	Settle bound expires	RollingBack or Finished	Kill remaining work and record interruption; Hold forbids rollback.
RollingBack	An undo returns, fails, crashes, or times out	RollingBack	Continue remaining entries; retain every located result.
Settling / RollingBack / Finished	Repeated cancellation	Unchanged	No new admission and no replacement of the original trigger.

3.7 Step action lifecycle

A retry creates another attempt while preserving prior action evidence.

Step progress waiting · attempting · compensating · retry-scheduled · succeeded · failed-step · interrupted · undoing · undone · undo-failed-step · skipped

Step occurrence

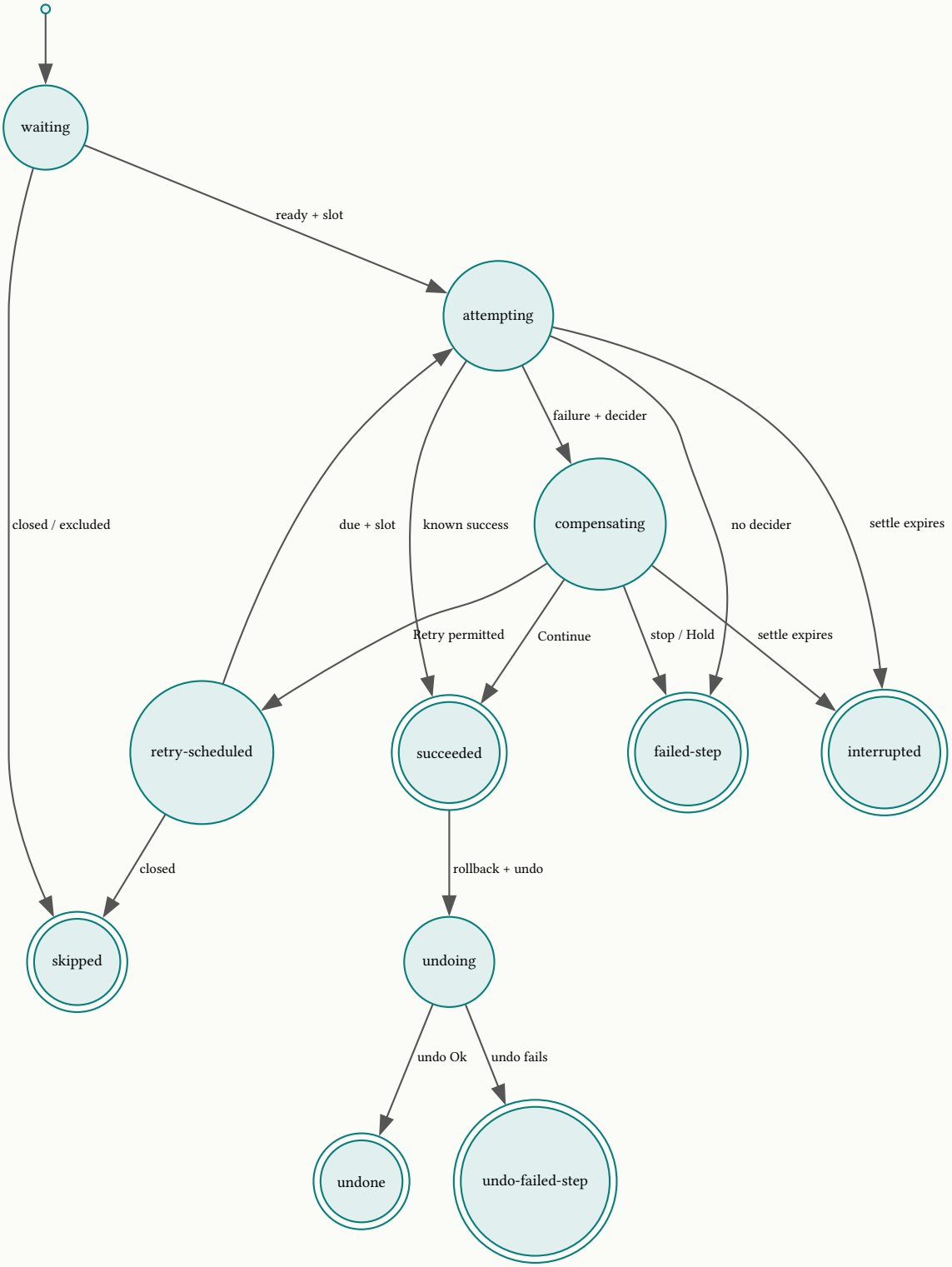
entity · stateful · workflow · owned by Saga coordinator

A structurally addressed operation within one Execution.

ATTRIBUTES

- derived **Address** · Step address
  Resolved from construction scope, name, and occurrence.
- derived **State** · Step progress step-progress · step-lifecycle
  Attempt numbers and correlated action sequences are associated data.
- derived **Attempts remaining** · Finite positive budget including the first attempt
  A compensation decision still runs after the last allowed attempt because cleanup may be needed.

STATE MACHINE Step progress and rollback



Succeeded remains terminal when no rollback is needed or no undo exists. FailedStep includes the stopped node of a held run; it does not imply that the effect is known.

- Public Progress reports phase and every StepProgress without native values or closures. It is a read-only observation, never a control or recovery token.

3.8 Compensation and uncertainty

Failed-attempt decisions and completed-step undo grant different permissions.

Recovery policy

responsibility

Choose how a failed attempt affects its own progress and prior completed work.

interface

FailedAttempt(input, failure, attempt, attempts_left, key) ⟶ Recovery(output, error, undo_error).

interactions

Receives returned native errors and native crash/timeout evidence; may perform cleanup under its own bound.

invariants

Retry does not erase uncertainty; Continue supplies the replacement output's undo; Hold grants neither retry nor rollback authority.

failure

A raising or timed-out decider retains unknown compensation evidence; AbortAfterCleanupFailure retains its cleanup error separately.

Decision	Permitted transition	Required retained evidence
Retry / RetryAfter	Another attempt only within budget and open admission; delay capped	Earlier unknown action stays recorded; retry-superseded differs from exhausted budget.
Continue(output, undo)	Accept replacement output and its undo capability	Replacement completion is known; original uncertain attempt remains unknown.
Abort(error)	Fail and permit reverse undo of known completions	Native application cause plus earlier unknown effects.
AbortAfterCleanupFailure(error, undo_error)	Same rollback permission	Separate compensation cleanup failure.
Hold(evidence)	End Unresolved after siblings settle; do not undo	Native held evidence and every completed step left in place.

- `unknown_when` marks returned business errors whose external effect may have happened. Without a deciding compensation, default `on_unknown(Reconcile)` preserves completed reservations; explicit `RollBack` permits their undo.
- Exhausted retry of a marked uncertain error follows the same Hold/RollBack policy. An explicit Abort permits rollback even after a marked error.
- Crashes and timeouts without a decider fail and roll back known completions while retaining the crashed/timed-out action as unknown. A decider's Abort may produce StepFailed after a crash; the cause alone cannot establish known effects.
- Applications own downstream idempotency and reconciliation evidence. Passing an idempotency key alone never proves provider deduplication or exactly-once effects.

BEHAVIOR

An uncertain retry cannot become clean completion

Given An attempt ended with unknown effect evidence.

When A later attempt or replacement reaches the workflow output.

Then The outcome retains the output and the earlier unknown effect in CompletedWithUnknownEffects.

level: boundary area: Unknown effects

BEHAVIOR

Held uncertainty preserves prior completions

Given A step returns uncertain evidence without rollback permission.

When The run settles.

Then The outcome is Unresolved and completed steps remain held.

Then No earlier completion is claimed as reversed.

level: boundary area: Compensation

- The uncertainty and rollback decisions are recorded in ADR-0003.

3.9 Outcome and settlement

One full typed report is the authority for application and operational evidence.

Outcome	Payload	Meaning
Completed	Native output	Output reached with no unknown actions.
CompletedWithUnknownEffects	Native output + nonempty unknown effects	Output reached; earlier effects still require reconciliation.
Failed	Cause(error) + Settlement(error, undo_error)	Failure stops admission and settlement records rollback evidence.
Cancelled	CancelRequested or OwnerExited + Settlement	Cancellation intent stopped admission; it does not assert complete reversal.
Unresolved	Step address + native evidence + Settlement	Held uncertainty stopped progress without rollback authority.

Settlement

value-object immutable workflow owned by Saga execution

The immutable evidence accompanying a stopped Outcome.

ATTRIBUTES

observed **Reversed and retained completions** · Undone × not-undoable × held step addresses
Known undo success is distinct from a completed effect lacking undo or held without permission.

observed **Failed cleanup** · All located undo and compensation failures
Native undo errors, crashes, and timeouts remain distinct; one failure never stops the remaining journal.

observed **Concurrent stopping evidence** · Interrupted steps × sibling causes
The primary trigger remains separate from failures arriving during settlement.

observed **Unknown effects** · Step × action × ending
Attempt and compensation number remain visible; identical addresses do not erase distinct actions.

Outcome projection

responsibility

Classify full reports conservatively and provide payload-safe summaries.

interface

`outcome.classify(report, explain)`, `kind`, `held_steps`, `failure_kind`, `describe_failure`, and `summary`.

interactions

A single private evidence projection supplies classification and rendering; the caller retains the original Outcome.

invariants

A definite stopped result requires every effect known and none left in place. Summary excludes output, business/undo error payloads, and crash reasons.

failure

Completion with uncertainty, Hold, retained effects, failed undo, or unknown actions classify unresolved. Neither definite classification nor reporting NotStarted chooses retry policy.

Projection	Permitted result	Disclosure
Clean Completed	Native output	No payload-safe summary promise applies to the returned output itself.
Fully reversed typed failure or cancellation	Definitely(message)	Caller explain renders selected business error.
CompletedWithUnknownEffects or retained/unknown effects	Unknown(evidence)	Complete safe settlement summary; Unresolved adds explicit held-error explanation.
summary	Outcome/cause kinds and located evidence	Never invokes explain; distinct actions/ attempts and all settlement categories remain.

- A safe summary keeps addresses visible. Typed cause accessors and caller-provided renderers may expose private application data; the application owns disclosure. ADR-0007 records why reporting retains full reports before projection.
- Each action of a run (a step attempt, a compensation decision, an undo) ends with a known result or an unknown effect. `unknown_effects(outcome)` lists every unknown effect as (step, action, ending) and is empty exactly when every effect is known. An action is recorded when it ends, so a later decision cannot hide it: a crashed or maybe-sent attempt retried to success yields CompletedWithUnknownEffects.
- A retry decided after the run began settling for another reason is reported as retry-superseded, never as an exhausted budget.
- Settling ends as soon as nothing is in flight. A settle window of zero rolls back at once and reports in-flight steps as interrupted. The window is set per run, never per cancellation.

3.10 Time and capacity

A duration bounds its named operation; it is not an end-to-end return promise.

Bound	Default	Ownership and limit
Concurrent attempts + compensation decisions	Online scheduler count	Per run; at least one; retries need slots.
Run deadline	Infinity	After(Duration) is opt-in; triggers settlement and rollback.
Attempt timeout	60 seconds	Step-specific timeout overrides default in either direction; Infinity opts out for undeclared steps.
Sibling settlement	5 seconds	Can be zero; no new admission; active siblings may finish.
Each compensation / undo	5 seconds	Positive; undo runs sequentially and is not retried.
RetryAfter cap	5 minutes	Nonnegative; requested larger delay clamps and is observed.
Coordinator startup handshake	5 seconds	Not configurable; separate from the public control handshake.
Observation wait (await, progress, testing wait)	Required caller Duration	A timeout limits the caller's wait and never cancels the run.
Synchronous run	Until an outcome or coordinator loss	A caller needing a bound uses start and await.
Receiver readiness	5 seconds	Proven prelaunch failure returns typed reporting error.
Reporting rollback / notification	Caller Duration	1 through 2^32−1 milliseconds; each callback has this bound.

Bound	Default	Ownership and limit
Sinal handlers	Synchronous in the coordinator	A slow handler delays coordinator processing; a forwarder route removes that coupling.

- Every timeout, deadline, interval, and lease is a Duration; no public function takes milliseconds as an integer. A bound that may be lifted takes After(Duration) or an explicit Infinity, so unbounded is never implicit. The storage, checkpoint, and drive bounds belong to the durability context.

FORMULA

Conservative sequential action budget

$$B = \sum_s A_s T_s + \sum_s A_s C + \sum_s A_s R + S + JC$$

NOTATION

A_s allowed attempts for step s
T_s effective finite attempt timeout
C cleanup timeout
R retry-delay cap
S settle timeout
J known journal entries requiring undo

The sum overcounts parallel work and the final retry delay. It bounds the configured action waits, not arbitrary synchronous observer or extension latency.

- A finite deadline substitutes its execution portion for the summed action portion, then settlement and cleanup still occur. Infinity, caller callbacks without an applicable timer, or synchronous Sinal handlers can defeat a literal wall-clock bound.
- Local progress/await bounds limit the caller's observation wait; a timeout does not cancel the run. The caller explicitly cancels and consumes the terminal report before dropping an ordinary Execution handle.
- The local start path contains an internal readiness wait and a separate public control-handshake wait, each five seconds. Their sum is not a total-start bound.
- Persistent deadlines and backoff use wall-clock timestamps across restart; run timers use monotonic timing. The deployment owns wall-clock suitability and cannot infer cross-node agreement from local Duration types.

BEHAVIOR

Cancellation settles known concurrent work

Given A local run has active siblings.

When Explicit cancellation or owner exit stops admission.

Then Active siblings may finish within the configured settle window.

Then Remaining actions are interrupted and unknown; only known completed effects enter rollback.

level: boundary area: Local lifetime

3.11 Local ownership and reporting

The starting process owns execution, while outcome delivery may belong to a survivor.

Local execution port

responsibility

Start one owner-bound coordinator and expose its outcome, cancellation, and progress.

interface

`run`, `start`, `await`, `start_reporting`, `cancel`, `progress`, `pid`, and `run_id`.

interactions

Monitors starting process; ordinary handle delivers to that owner, reporting handle delivers once to its Subject.

invariants

Only the ordinary starter may await. Reporting handles cannot await; cancel/progress work from any process.

failure

InvalidConfig starts no work; ExecutionLost cannot prove no effect started. Await distinguishes timeout, NotOwner, consumed outcome, and typed Lost evidence.

- A successful ordinary await consumes the outcome and drains its monitor. A second await may first time out while the coordinator is exiting; AlreadyAwaited is guaranteed only after exit.
- A dropped handle after a timed-out await can leave its terminal outcome or monitor message in the owner mailbox. Cancellation requires a subsequent await to consume completion.
- `start_reporting` allocates no outcome monitor/mail in the starting process. Its Subject receives at most one terminal report after settlement; coordinator death without a report remains distinguishable through monitoring the coordinator pid, whose Down message always arrives after its outcome.

3.11.1 One invocation receiver

Owned reporting

responsibility

Preserve a full typed report across abnormal exit of one invocation owner.

interface

`reporting.run_owned(workflow, input, config, on_stopped, rollback_within)` returns Result(Outcome, reporting.Error); callback receives the same result.

interactions

Starts an independent receiver and waits for readiness before execution; receiver monitors invocation and coordinator; a guarded worker invokes on_stopped.

invariants

Receiver lives until invocation exit even after synchronous return. Normal owner exit produces no second notification; abnormal exit can deliver settlement.

failure

Startup exit/timeout, admission, receiver loss, and coordinator loss retain typed operational causes; callback crash/timeout leaves notification unconfirmed.

- Use this boundary within a per-invocation worker. Repeated use from a long-lived actor retains one receiver per invocation until that actor exits.
- rollback_within bounds owner-loss waiting when no coordinator was reported and bounds the notification worker. Once a known coordinator exists, its configured settlement/cleanup govern completion; rollback_within is not a global execution deadline.
- Readiness failure closes the startup reply channel, removes monitors, and preserves unrelated mail before any workflow launch. Normal result return and admission failure remove the caller's receiver monitor.
- NotStarted is proven only for rejected rollback/configuration or prelaunch receiver failure. Lost execution handshake, receiver loss, and coordinator loss are Unknown; typed run_error/exit_reason accessors retain causes and safe describe_error excludes crash payloads.

3.12 Observation and verification

Observations describe transitions; oracle and benchmark evidence state exactly what they compare.

Telemetry

responsibility

Emit package-owned lifecycle facts after corresponding transitions.

interface

Six typed event descriptors: run_started, run_stopped, step_started, step_stopped, compensation_stopped, undo_stopped.

interactions

Sinal transports typed measurements and metadata; step clients read the same correlation from EffectKey. Metadata carries the workflow name, this VM's run id, the durable execution id when one exists, and the correlation.

invariants

Every event carries a correlation; no telemetry value drives workflow state. A compensation event reports the retry delay a RetryAfter decision scheduled and whether the cap shortened it.

failure

Handler raises are isolated by Sinal. Synchronous slow handlers delay coordinator processing and therefore timing; no guaranteed durable observation delivery is implied.

- The local correlation defaults to a unique value; the first durable drive saves an explicit handle correlation or the execution-id-derived default. Later handles cannot change it; ADR-0006 records the identity contract.
- Saga emits no separate claim, release, lease, or checkpoint event family. Applications obtain those facts from durable and storage results, and never interpret a missing event as absence of an effect.
- A route for the saga event prefix to a Sinal forwarder moves handler work out of the coordinator.

Run synchronization for tests

responsibility

Let a test wait for an observable run state without a guessed sleep.

interface

`testing.wait_until(execution, matching, within)` polls public Progress until the predicate accepts a snapshot or the overall bound elapses.

interactions

Polling uses a short internal interval and the monotonic clock for its deadline. A consumer test supplies its own blocking gate for step bodies; the order consumer's test support holds the maintained gate.

invariants

Progress is a read-only observation and never a control or recovery token.

failure

WaitTimedOut means the predicate never matched in time. RunEnded means the coordinator was already gone.

- A poll sees only the snapshot current at that instant, so a quickly passing state can be missed. A predicate accepts its target state or any state beyond it: waiting for a step in compensation also accepts a scheduled retry.
- Saga ships no gate for a step body to block on, because that is generic process coordination. A gate registers a reply subject owned by the entering process, because a step body runs in a fresh task process on every attempt, and it queues a release that arrives before any waiter.

3.12.1 Reactor oracle

- Reactor 1.0.6 is the pinned behavioral oracle for scheduling and rollback. The pin is a hex-locked dependency in the oracle project plus the upstream commit recorded in ADR-0010. Saga's native contracts, not Reactor's API, define the product.
- A scenario runs against live Reactor and against Saga, normalizes both outputs to one language-neutral text format, and compares them mechanically. A claimed match must produce an empty difference. A claimed deliberate difference must byte-match a checked-in expected difference. A change in either difference fails the check instead of silently reclassifying behavior.
- Captured raw Reactor output is retained separately as evidence of the Reactor side and is verified against a live run; it is not the input to the comparison.
- An expected difference is recomputed after a reviewed behavior change, never edited by hand.
- Seven scenarios are compared: sequential chain (D1), undo after a failing fourth step (D2), a slow, a fast-failing and a quick sibling (D3), two failures then success (D4), compensation that continues with a replacement (D5), ten independent steps under concurrency three (D6) and a diamond with one producer (D7). Two differences are deliberate: Saga undoes in reverse completion order (D2) and settles siblings, undoing a late completion once it is known (D3). The rest match.
- Halt and resume, abandoned steps, whole-run undo, dynamic steps, the DSL transformer suite, and graph export are not compared; the composition extension contracts own them. The decision is ADR-0010.
- The pin, the mapping of each adapted upstream test, every compared fact and the limits of the claim are retained in the oracle reference.

3.12.2 Run-cost benchmark

- The harness is a separate package that measures definition and execution through public modules. It measures a chain, a shared producer with fan-out and fan-in, and a sliding window of four dependency reads, at sizes from 10 to 2000 steps.
- Each cell measures definition once, one separate warm-up, then 20 runs below 1000 steps or 15 runs at larger sizes. Monotonic timings are microseconds with nearest-rank median and p95.
- Every timed run must complete with the shape's expected output, and an atomic counter checks builder invocations across definition and runs. A cell whose warm-up exceeds 60 seconds is skipped, and larger sizes of that shape stop.
- Trivial integer steps isolate graph and scheduling cost. The harness establishes no remote-effect latency, rollback cost, storage throughput, memory limit, or production tail latency, and a doubling ratio supports a cost hypothesis rather than an asymptotic bound. The decision is ADR-0011.
- The run command, retained results, capture caveats and limits are retained in the benchmark reference results are observational and no timing threshold fails a gate.

3.12.3 Usage examples

- Every documented usage topic exists as compiled, runnable code in the order consumer, one module per topic, and each module asserts what it shows. The full gate runs the consumer tests and the consumer program, so an API change that breaks an example fails the build.
- The README keeps a short usage section that mirrors the examples. No prose usage guide exists; ADR-0014 records the decision.
- Operating guidance for durable runs is expressed as examples: a stable id that reconnects to the saved run, error kinds mapped to application actions, discovery of unfinished runs, and recovery of a committed local effect through an attempt resolver. The durability contracts they exercise are owned by the durability context.
- The topic map is in the order consumer README.

3.13 Composition extension contracts

Extension capabilities carry explicit ownership instead of changing the static facade's meaning.

3.13.1 Value-dependent traversal and local continuation

Typed expansion

responsibility

Compose known native types after a value arrives and traverse a homogeneous runtime collection.

interface

A value-dependent composition and traversal surface over the canonical Workflow; no signatures are accepted.

interactions

Expands typed local fragments; durable fragments need stable factory identity/version and saved branch selection.

invariants

Ordering, graph-size, cancellation, and shared/partitioned capacity must be explicit. Arbitrary heterogeneous steps and named native-result lookup remain excluded.

failure

Unsupported durable local closures are refused at eligibility; runtime collection/resource bounds must fail explicitly before uncontrolled expansion.

- Explicit local halt/resume retains typed state and control authority. Copyable execution values must not permit repeated external effects merely because a completion or wake token matches.
- A paused or blocked recovery retains native input, original cause, continuation, and completed journal. Retry permission, cancellation/rollback permission, and reconciliation authority remain separate; relative wake tokens reject stale or duplicate delivery. Hold is a terminal report, never a resumable recovery surface.

3.13.2 Approvals and historical review

Approval continuation

responsibility

Suspend at an authenticated answer boundary without treating waiting as business failure.

interface

Retained input, wait identity/revision, authenticated answer, command identity, deadline, and explicit continuation owner.

interactions

Commits waiting state and releases worker capacity; Fabric owns agent approval policy while Saga owns its workflow boundary.

invariants

Stale/duplicate signals cannot resume another revision. Forking from history cannot silently reuse compensation authority over prior effects.

failure

Expired, rejected, cancelled, stale, incompatible, corrupt, and ambiguous-effect outcomes stay distinct. Prior-effect policy for forks remains a ruling.

3.13.3 Independent children and compensation authority

Child boundary

responsibility

Attach an independently owned execution without flattening its journal into its parent.

interface

Typed child contract with actual input/output/error/undo-error codecs; prepared command; checked admission receipt; observation and cancellation intent.

interactions

Parent persists route, exact request, stable command, and attachment; after restart it reattaches to compatible child owner state.

invariants

Compatibility and authority precede decoding. Waiting releases capacity needed by child; parent and child recovery remain distinct.

failure

Unknown/invalid admission preserves exact request for reconciliation; later failed retry does not resolve earlier uncertainty. Missing owner/run, pruned output, incompatibility, and corrupt result differ.

Compensation authority

responsibility

Authorize one logical compensation operation for a retained successful execution.

interface

Owner-backed exclusive claim, durable receipt reference, typed rebound undo-error codec, stable request command, and separate compensation observation.

interactions

Owner checks success, journal retention, claimant identity, and current claim; parent retains attachment and receipt references.

invariants

Success alone grants no authority. Copyable handles never enforce single use; storage admits at most one logical operation per claim and replays its original admission.

failure

Retention expiry/pruning, competing claim, unauthorized caller, unavailable owner, incompatible contract, uncertain admission, and undo failure remain distinct.

Independent state axis	States	Rule
Historical child result	Pending / completed / failed / unresolved; output may be pruned	Compensation does not rewrite historical success.
Compensation operation	Pending / running / succeeded / failed / unresolved	One logical admission per authority claim.
Result and journal retention	Available / expired / pruned	A retained journal may outlive output; replayed admitted request retains identity after fresh-admission expiry.

- These distinctions are contracts. Concurrent storage enforcement, authentication, actual compensation, restart, and local post-success authority are the build boundary recorded in the pending ledger.

3.14 Runtime-authored graphs

Finite data contracts type runtime wiring without pretending to create new native Gleam types.

Runtime graph compiler

responsibility

Turn an editable graph into a checked immutable executable definition.

interface

Versioned node registry, parametric port expressions, configuration contracts, draft graph, publication errors, and PublishedGraph.

interactions

Blueprint owns finite runtime contracts/validated values; Saga owns registration, instantiation, wiring, graph validation, and execution.

invariants

Only PublishedGraph executes. Runtime data remains opaque validated values; static adapters retain actual native codecs.

failure

Unsupported schemas, unresolved versions/types, invalid configuration, illegal connections, cycles, missing inputs, and resource limits are located publication failures.

Runtime graph status draft published retired

Runtime graph definition

aggregate stateful workflow owned by Saga runtime graph compiler

The runtime-authoring model with editable and immutable states.

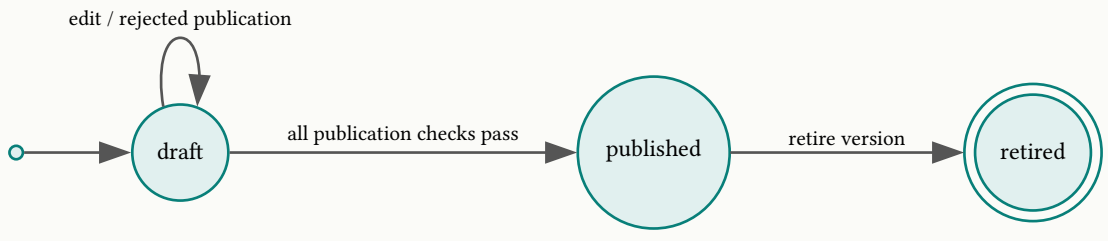
ATTRIBUTES

derived **Status** · Runtime graph status runtime-graph-status · runtime-graph-lifecycle
Publication derives executable capability; retirement and existing-run retention require an explicit policy before implementation.

authored **Node contract references** · Registry kind × version × finite port contracts
Deployed implementations remain separate from serializable graph data.

derived **Resolved schema bindings** · Concrete finite runtime contracts
Publication resolves every variable and validates values/connections before execution.

STATE MACHINE Runtime graph publication



A changed executable graph is a new version derived from a draft. Retirement states how active executions and registry retention are treated.

3.14.1 Parametric node model

Family	Type relation
Identity / Delay	T ⟶ T
Choose	Bool × T × T ⟶ T

Family	Type relation
Collect / Constant / Validate	T… ⟶ Array(T); () ⟶ T; wire value ⟶ validated T
Map	Array(A) × Graph(A, B) ⟶ Array(B)
LLM	Input ⟶ Output under declared finite contracts

- A node definition retains kind/version, input/output port definitions, configuration schema, and deployed implementation. Type expressions include concrete contract, variable, array, optional, and finite object.
- Instantiation binds variables through incoming connections, including array/optional/object structure. The first compatibility rule is exact structural equality with explicit transformation nodes; width subtyping and broader assignability require separate acceptance.
- Every runtime port has an opaque identity and validated contract; every node output is validated against its output contract. Required field access derives the field contract and returns a located failure.
- A native adapter validates and decodes input before its native callback, then checks encoding and validates output. A custom codec with unavailable schema cannot silently claim compatibility.
- The LLM node composes llm_wire provider execution and Blueprint output validation. Fabric can supply bounded agent policy; it does not become Saga's provider transport.

Runtime node execution

responsibility

Execute a published node through its checked concrete port contracts.

interface

Opaque port identifiers and maps of Blueprint validated values; concrete input/output contracts and located node errors. Native adapters decode typed input, invoke their original handler, encode output, and validate the encoded result.

interactions

Saga schedules the registered implementation; Blueprint checks contract values; application services own actual remote effects and reconciliation.

invariants

A successful node output conforms to its declared contract. Registry identity and publication remain fixed for one execution; a dictionary never permits unchecked native-value lookup.

failure

Input decoding, native handler failure, output encoding, invalid node output, timeout, and uncertain remote effects remain distinct. Publication does not prove external provider deduplication.

3.14.2 Publication and hostile input

Publication stage	Required acceptance
Configuration and registry	Decode every configuration; resolve definition/version; validate constants and defaults.
Type derivation	Instantiate parameters; unify connected ports; derive output schemas.
Graph structure	Detect cycles; reject missing producers and undeclared duplicate writers; require associative declared reducers.
Boundary completeness	Check required inputs; validate graph input/output contracts; reject unreachable outputs.
Executable publication	Produce one immutable PublishedGraph with all type variables resolved.

- Finite contracts forbid cyclic references for this surface; arrays and nested objects still remain finite schema trees. Schema depth/size, graph nodes/edges, traversal, and execution resource budgets need explicit bounds before hostile-input acceptance.
- Serialization saves graph data and versioned implementation identity, never closure code. Restore checks registry versions and schema/value compatibility before execution.
- Visual editors, diagnostic metadata, exported graphs, hostile publication tests, and persistence/recovery scenarios belong to this surface; static Port compiler tests do not verify it.

Glossary

Every term the layer declares, with the context that owns it.

Approval

owned by execution

A suspended decision boundary retaining input and requiring an authenticated, revision-checked, deduplicated answer.

Attempt

owned by execution

One invocation of a step under its positive attempt number and fixed attempt budget.

Checkpoint

owned by durability

A complete saved data representation of durable execution progress and its recovery authority.

Child

owned by execution

An independently owned workflow execution attached to a parent through an explicit admission and observation contract.

Claim

owned by durability

The revocable execution ownership value identified by execution id, increasing generation, and adapter token. *Avoid*: process identity — a current claim can be used by another process.

Codec

owned by durability

A versioned pair of fallible conversions between a native value and saved text.

Compatibility stamp

owned by durability

A computed identity of the workflow structure, semantic versions, codecs, and declared recovery capabilities used to admit saved state.

Compensation

owned by execution

The failed-attempt decision that selects retry, replacement output, abort, or held uncertainty. *Avoid*: undo — undo addresses a known completed step.

Compensation authority

owned by execution

The explicit owner-backed right to admit compensation of a successful child or completed execution.

Conformance suite

owned by durability

The public storage check that an adapter satisfies atomic creation, claims as values, ownership loss, stale writes, the cancellation race, listing, release, and killed-runner recovery.

Correlation

owned by execution

The Sinal identity carried by an execution's action contexts and observations for joining application evidence.

Drive

owned by durability

One bounded runner attempt to advance a durable execution until it finishes, suspends, loses ownership, or exhausts its execution budget.

Durable execution

owned by durability

One saved execution identified within a storage and recoverable through a compatible workflow definition.

Effect key

owned by execution

An opaque action context containing stable idempotency identity, attempt identity, attempt number, and execution correlation.

Execution

owned by execution

One interpretation of a workflow for one input, with its own action progress, completed values, journal, and terminal evidence.

Execution handle

owned by execution

An opaque reference to a running local coordinator and its outcome-delivery ownership.

Fencing

owned by durability

The rule that every saved write names the current claim generation and token, so a write by a replaced owner is refused.

Hold

owned by execution

A compensation decision retaining application evidence without authorizing retry or rollback of completed steps.

Idempotency key

owned by execution

The effect-key component that is identical for every attempt of one step action and distinct for an undo, passed to a downstream service so that it can deduplicate. Passing it never proves that the service deduplicated. *Avoid*: request id — an id says nothing about repeated attempts.

Lease

owned by durability

A time-limited claim whose expiry permits takeover under an adapter's policy. Expiry never retracts an external call that an earlier runner already sent.

Local run

owned by execution

One execution of a workflow whose lifetime belongs to the process that starts it.

Oracle

owned by execution

A pinned external implementation whose observable behavior in named scenarios is compared mechanically with Saga's, where each difference is either empty or checked in.

Outcome

owned by execution

The typed terminal report of completion, completion with unknown effects, failure, cancellation, or unresolved held evidence.

Persistence capability

owned by durability

The checked association of an existing workflow with root codecs, compatibility identity, and checkpoint limits.

Port

owned by execution

An opaque typed reference to a dependency value within one workflow construction. *Avoid*: result name — names identify steps but do not transport native values.

Published graph

owned by execution

An immutable executable runtime graph with resolved node versions, type parameters, configurations, and checked connections.

Reconciliation

owned by durability

The application establishing from downstream evidence whether an interrupted admitted action took effect. *Avoid*: retry — a retry repeats the action instead of establishing its result.

Report receiver

owned by execution

The independent process retaining outcome-delivery responsibility for one short-lived invocation owner.

Resolver

owned by durability

An application callback that establishes the result or absence of an interrupted admitted action without blindly repeating that action.

Revision

owned by durability

The increasing count of committed checkpoint changes for one durable execution.

Run handle

owned by durability

An opaque durable reference combining a persistence capability, storage, execution identifier, and drive configuration.

Runtime graph

owned by execution

A graph authored after native compilation whose values and connections are checked against finite runtime data contracts.

Settlement

owned by execution

The complete retained evidence about undo, remaining known effects, interrupted actions, sibling failures, and unknown effects after admission stops.

Step

owned by execution

A typed operation definition with its attempt, recovery, undo, timing, and optional persistence capabilities.

Step address

owned by execution

The structural identity of a step occurrence: nested scope, name, and positive occurrence rank within that scope and name.

Step occurrence

owned by execution

One structurally addressed use of a reusable step inside a workflow definition and execution.

Storage

owned by durability

The atomic persistence port for saved bytes, execution claims, conditional commits, cancellation intent, and unfinished-execution discovery.

Time bound

owned by execution

A Duration limiting one named operation, written as After a Duration or as an explicit Infinity. A time bound never promises an end-to-end return time.

Undo

owned by execution

An application action intended to reverse a known completed step using its retained input, output, and own effect key.

Undo journal

owned by execution

The ordered completed-step records that retain undo capability and actual completion order.

Unknown effect

owned by execution

An action whose effect status is unresolved because it crashed, timed out, was interrupted, or returned an application error classified as uncertain.

Workflow

owned by execution

A reusable typed definition of dependency-connected steps with one input, output, business-error, and undo-error contract.